

1.

```
#include<stdio.h>

int main()
{
    printf("Hello Linux\n");

    return 0;
}
```

```
mahima@Mahima:~$ nano hello.c
mahima@Mahima:~$ gcc hello.c -o hello
mahima@Mahima:~$ ./hello
Hello Linux
```

2.

```
#include<stdio.h>

int main()
{
    char name[30];

    printf("Enter your name: ");
    scanf("%s",name);

    printf("Welcome %s\n",name);

    return 0;
}
```

```
mahima@Mahima:~$ nano username.c
mahima@Mahima:~$ gcc username.c -o username
mahima@Mahima:~$ ./username
Enter your name: mahima
Welcome mahima
```

3.

```
#include<stdio.h>

#include<unistd.h>


int main()
{
    printf("Process ID = %d\n",getpid());

    return 0;
}
```

```
mahima@Mahima:~$ nano process.c
mahima@Mahima:~$ gcc process.c -o process
mahima@Mahima:~$ ./process
Process ID = 654
```

4.

```
#include<stdio.h>

#include<unistd.h>

int main()
{
    fork();

    printf("Hello\n");

    return 0;
}
```

```
mahima@Mahima:~$ nano fork.c
mahima@Mahima:~$ gcc fork.c -o fork
mahima@Mahima:~$ ./fork
Hello
Hello
```

5.

```
#include<stdio.h>

#include<unistd.h>


int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        printf("I am Child Process\n");
    }
    else
    {
        printf("I am Parent Process\n");
    }
    return 0;
}
```

```
mahima@Mahima:~$ nano parentchild.c
mahima@Mahima:~$ gcc parentchild.c -o parentchild
mahima@Mahima:~$ ./parentchild
I am Parent Process
I am Child Process
```

6.

```
#include<stdio.h>
#include<unistd.h>

int main()
{
    pid_t pid;
    pid=fork();

    if(pid==0)
    {
        printf("Child PID = %d\n",getpid());
        printf("Parent PID = %d\n",getppid());
    }
    else
    {
        printf("Parent PID = %d\n",getpid());
    }

    return 0;
}
```

```
mahima@Mahima:~$ nano parentchildpid.c
mahima@Mahima:~$ gcc parentchildpid.c -o parentchildpid
mahima@Mahima:~$ ./parentchildpid
Parent PID = 828
Child PID = 829
Parent PID = 828
```

7.

```
#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>

int main()
{
    pid_t pid;
    pid=fork();
    if(pid==0)
    {
        printf("Child Running\n");
    }
    else
    {
        wait(NULL);
        printf("Parent Running After Child\n");
    }
    return 0;
}
```

```
mahima@Mahima:~$ nano wait.c
mahima@Mahima:~$ gcc wait.c -o wait
mahima@Mahima:~$ ./wait
Child Running
Parent Running After Child
```

8.

```

#include<stdio.h>

#include<unistd.h>

int main()
{
    printf("Before exec()\n");

    execlp("ls", "ls", NULL);

    printf("After exec()\n");

    return 0;
}

```

```

mahima@Mahima:~$ nano waitt.c
mahima@Mahima:~$ gcc waitt.c -o waitt
mahima@Mahima:~$ ./waitt
Before exec()
a.out      filename.c  fork      hello      parentchild  parentchild.c.save  parentchildpid.c  process.c  username.c  wait  waitt
filename  filename.c.save  fork.c    hello.c    parentchild.c  parentchildpid      process          username  username.c.save  wait.c  waitt.c

```



```
#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        execlp("date","date",NULL);
    }
    else
    {
        wait(NULL);

        printf("Child Completed\n");
    }
    return 0;
}
```

```
mahima@Mahima:~$ nano forkexec.c
mahima@Mahima:~$ gcc forkexec.c -o forkexec
mahima@Mahima:~$ ./forkexec
Tue Aug  4 07:11:44 UTC 2026
Child Completed
```